Week 1 Progress Report

Network Quality Monitoring System - Foundation and Data Model

Prepared by	Zohaib Akmal
Report date	28 October 2026
Plan period (Week 1)	19 - 25 October 2026
Week 1 completed	28 October 2026 (3 days after the planned end date)
Repository	github.com/L1S24BSSE0071/Network-Quality-Monitoring-System
Version tag	week-1

1. Summary

The Week 1 milestone has been met: a monitoring target can be added through the API, a check can be run by hand against it, and the result is stored in the database. All Week 1 deliverables are complete, committed and tagged **week-1**. Week 1 finished three days later than planned, because work began late. The work was completed in plan order and the remaining Week 2 tasks follow next (see section 9).

2. Milestone

"A target can be added, one check can be run by hand and its result appears in the database." Status: **Met**.

3. Tasks against the plan

Planned task	Status	Output
Requirements and scope document	**Done**	docs/requirements.md: 22 functional requirements, 6 validation rules, 9 non-functional requirements, exclusions and authorization statement
Database schema and system architecture	**Done**	docs/design.md: ER diagram, architecture diagram, check-flow and state diagrams (render on GitHub)
Repository, virtual environment, FastAPI skeleton, SQLite	**Done**	Repository on GitHub; pinned requirements.txt; app/main.py, app/database.py
Target API (create, list, edit, delete) with input validation	**Done**	app/routers/targets.py, app/schemas.py; documented at /docs
HTTP check with timeout and error handling; store one result	**Done**	app/checker.py; POST /targets/{id}/check saves to check_results
README draft and review notes	**Done**	README.md; docs/week1-review.md

4. Deliverables

Deliverable	Location in repository
Requirements and scope document	docs/requirements.md
ER and architecture diagrams	docs/design.md
Working target API	app/routers/targets.py
First stored check result	check_results table, via POST /targets/{id}/check
README draft	README.md
Review notes	docs/week1-review.md

5. Definition of done

Criterion	Result	Evidence
The API starts without errors	**Met**	Server starts; /health returns ok; also verified from a fresh clone of the repository using only the README steps
Invalid data returns a clear error message	**Met**	422 response lists the field and the reason (example in section 7)
Stored check result matches the manual check	**Met**	Result read back from the database matches the response returned by the check
Schema and architecture diagrams are in the repository	**Met**	docs/design.md
Work is committed and tagged week-1	**Met**	Feature branches merged into main; tag week-1 pushed

6. Implementation and testing

Technology: Python, FastAPI, SQLAlchemy, SQLite and httpx. **Design:** separate modules for database, models, schemas, checker and routers; each feature was developed on its own branch and merged into main after testing.

Validation rules implemented: name required and not blank; HTTP target must be a well-formed http:// or https:// URL; ping target must be a host name or IP address; interval between 10 and 3600 seconds; warning threshold lower than critical threshold. Rules are re-checked when a target is edited.

Check failures handled without crashing: timeouts, DNS failures, refused connections, invalid URLs, redirect loops, and expired or self-signed certificates. Each is stored with a readable error message. HTTP responses with status 400 or above are recorded as down, with latency and status kept.

Test area	What was tested	Result
Input validation	17 valid and invalid target inputs	All accepted or rejected as expected
Target API	19 scenarios: create, list, get, edit, delete, unknown id, cross-field rules on edit	All passed
HTTP checker	200, redirect, 404, 503, timeout, redirect loop, refused connection, DNS failure, self-signed and expired certificates, live website	All produced the expected outcome and message
End to end	Check stored, listed with a limit, removed when the target is deleted	Passed

Test area	What was tested	Result
Fresh install	Clone, install requirements, start, add target, run check	Passed

Testing so far used manual scripts; automated unit tests are planned for Week 2.

7. Demonstration evidence

Add a target - `POST /targets`

```
{"name": "Example", "host_or_url": "https://example.com"}
```

Run a check - `POST /targets/1/check` (result stored in the database)

```
{"id": 1, "target_id": 1, "checked_at": "2026-10-28T13:52:39",
 "is_up": true, "latency_ms": 171.76, "http_status": 200,
 "error_message": null}
```

Failed check (unresolvable host) is still stored

```
{"is_up": false, "latency_ms": null, "http_status": null,
 "error_message": "DNS lookup failed (host name could not be resolved)"}
```

Invalid input - warning threshold above critical threshold (status 422)

```
{"detail": [{"field": "(request)",
             "message": "warning_ms must be lower than critical_ms"}]}
```

8. Problems encountered and decisions

Problem or decision	Resolution
Week 1 finished three days after its planned end date because work started late.	Tasks were completed in plan order, one at a time, on 28 October. Week 2 is now the priority.
Measured latency included HTTP client start-up (about 760 ms for a local server), which would have made results inaccurate.	Found during testing and fixed: the timer now covers only the request. A local check now reads about 4 ms.
HTTP status 400 or above: up or down?	Decision: counted as down, but latency and status code are still stored.
Ping check	Ping targets can be created, but checking one returns a clear "not supported yet" response. Ping is optional in the plan and is added only if time allows.

9. Plan for Week 2 (running late)

- Scheduler: each enabled target is checked in the background at its own interval, with results stored and timestamped.
- Make sure timeouts, DNS failures, invalid URLs and certificate errors never stop the scheduler.
- Threshold evaluation: warning, critical and recovery events created only on state changes.
- Uptime percentage and history, events and summary endpoints for the dashboard.
- Unit tests for the threshold logic.
- Ping check only if time allows; per the plan, optional items are dropped first if the schedule is tight.